

```

In [3]: # =====
# Rank-Order Doctor-Hospital Assignment
#
# Main method:
#   Hungarian / Kuhn-Munkres Algorithm
#
# Baseline:
#   Sequential Greedy Assignment
#
# Evaluation:
#   1. Total rank cost
#   2. Average assigned rank
#   3. First-choice rate
# =====

# -----
# 1. INPUT DATA
# -----

doctors = [
    "D1", "D2", "D3", "D4", "D5", "D6",
    "D7", "D8", "D9", "D10", "D11", "D12",
    "D13", "D14", "D15", "D16", "D17", "D18"
]

# Each doctor's preference list:
# first item = rank 1
# second item = rank 2
# ...
# sixth item = rank 6

preferences = {
    "D1": ["A", "F", "D", "C", "E", "B"],
    "D2": ["A", "C", "B", "D", "F", "E"],
    "D3": ["E", "F", "A", "C", "B", "D"],
    "D4": ["A", "F", "E", "C", "D", "B"],
    "D5": ["E", "A", "F", "B", "C", "D"],
    "D6": ["B", "A", "F", "E", "C", "D"],
    "D7": ["E", "D", "B", "C", "F", "A"],
    "D8": ["A", "E", "C", "F", "B", "D"],
    "D9": ["A", "E", "B", "D", "C", "F"],
    "D10": ["B", "A", "D", "E", "C", "F"],
    "D11": ["E", "B", "C", "D", "F", "A"],

```

```

    "D12": ["D", "A", "C", "B", "F", "E"],
    "D13": ["D", "B", "C", "F", "A", "E"],
    "D14": ["C", "A", "F", "D", "B", "E"],
    "D15": ["C", "E", "A", "D", "B", "F"],
    "D16": ["D", "E", "B", "C", "A", "F"],
    "D17": ["C", "D", "E", "A", "B", "F"],
    "D18": ["B", "E", "A", "C", "D", "F"]
}

# Hospital capacities
capacities = {
    "A": 3,
    "B": 3,
    "C": 3,
    "D": 3,
    "E": 3,
    "F": 3
}

# -----
# 2. VALIDATE INPUT
# -----

def validate_input(doctors, preferences, capacities):

    hospitals = list(capacities.keys())

    # Check every doctor has a preference list
    for doctor in doctors:
        if doctor not in preferences:
            raise ValueError(
                f"Missing preference list for {doctor}"
            )

    # Check every doctor ranks every hospital exactly once
    for doctor in doctors:

        pref = preferences[doctor]

        if len(pref) != len(hospitals):
            raise ValueError(
                f"{doctor} does not rank all hospitals."
            )

```

```

        if set(pref) != set(hospitals):
            raise ValueError(
                f"{doctor}'s preference list is invalid."
            )

# Check total capacity
total_capacity = sum(capacities.values())

if total_capacity < len(doctors):
    raise ValueError(
        "Total hospital capacity is not sufficient."
    )

print("Input validation passed.")
print(
    f"Doctors: {len(doctors)}, "
    f"Hospitals: {len(hospitals)}, "
    f"Total capacity: {total_capacity}"
)

```

```

# -----
# 3. CREATE HOSPITAL SLOTS
# -----

```

```

def create_slots(capacities):

    slots = []

    for hospital, capacity in capacities.items():

        for slot_number in range(1, capacity + 1):

            slot_name = f"{hospital}{slot_number}"

            slots.append(
                {
                    "slot": slot_name,
                    "hospital": hospital
                }
            )

    return slots

```

```

# -----

```

```
# 4. CREATE RANK LOOKUP
```

```
# -----
```

```
def create_rank_lookup(preferences):  
    rank_lookup = {}  
  
    for doctor, pref_list in preferences.items():  
        rank_lookup[doctor] = {}  
  
        for rank, hospital in enumerate(  
            pref_list,  
            start=1  
        ):  
            rank_lookup[doctor][hospital] = rank  
  
    return rank_lookup
```

```
# -----
```

```
# 5. BUILD COST MATRIX
```

```
# -----
```

```
def build_cost_matrix(  
    doctors,  
    slots,  
    rank_lookup  
):  
    cost_matrix = []  
  
    for doctor in doctors:  
        row = []  
  
        for slot in slots:  
            hospital = slot["hospital"]  
  
            cost = rank_lookup[doctor][hospital]  
  
            row.append(cost)  
  
        cost_matrix.append(row)
```

```
return cost_matrix
```

```
# -----  
# 6. PRINT COST MATRIX  
# -----
```

```
def print_cost_matrix(  
    doctors,  
    slots,  
    cost_matrix  
):
```

```
    print("\nCOST MATRIX")  
    print("=" * 90)
```

```
    slot_names = [  
        slot["slot"]  
        for slot in slots  
    ]
```

```
    header = "Doctor".ljust(8)
```

```
    for slot in slot_names:  
        header += slot.rjust(4)
```

```
    print(header)  
    print("-" * len(header))
```

```
    for doctor, row in zip(  
        doctors,  
        cost_matrix  
    ):
```

```
        line = doctor.ljust(8)
```

```
        for cost in row:  
            line += str(cost).rjust(4)
```

```
        print(line)
```

```
# -----  
# 7. HUNGARIAN ALGORITHM  
#
```

```

# Minimum-cost assignment implementation.
#
# This is a Kuhn-Munkres / Hungarian algorithm implementation
# using dual potentials and augmenting paths.
#
# cost_matrix must have:
#     number of columns >= number of rows
#
# -----

```

```

def hungarian_algorithm(cost_matrix):

    n = len(cost_matrix)
    m = len(cost_matrix[0])

    if n > m:
        raise ValueError(
            "Hungarian algorithm requires "
            "at least as many slots as doctors."
        )

    # Potentials
    u = [0] * (n + 1)
    v = [0] * (m + 1)

    # p[j] = doctor currently matched to column j
    p = [0] * (m + 1)

    # Previous column in augmenting path
    way = [0] * (m + 1)

    for i in range(1, n + 1):

        p[0] = i

        j0 = 0

        minv = [float("inf")] * (m + 1)

        used = [False] * (m + 1)

        while True:

            used[j0] = True

            i0 = p[j0]

```

```

delta = float("inf")

j1 = 0

for j in range(1, m + 1):
    if not used[j]:
        current_cost = (
            cost_matrix[i0 - 1][j - 1]
            - u[i0]
            - v[j]
        )

        if current_cost < minv[j]:
            minv[j] = current_cost
            way[j] = j0

            if minv[j] < delta:
                delta = minv[j]
                j1 = j

    # Update potentials
    for j in range(0, m + 1):
        if used[j]:
            u[p[j]] += delta
            v[j] -= delta
        else:
            minv[j] -= delta

    j0 = j1

    # Empty column found
    if p[j0] == 0:
        break

# Augment matching
while True:

```

```

        j1 = way[j0]

        p[j0] = p[j1]

        j0 = j1

        if j0 == 0:
            break

# assignment[i] = column assigned to row i
assignment = [-1] * n

for j in range(1, m + 1):

    if p[j] != 0:

        doctor_index = p[j] - 1
        slot_index = j - 1

        assignment[doctor_index] = slot_index

return assignment

```

```

# -----
# 8. CONVERT HUNGARIAN RESULT TO DOCTOR -> HOSPITAL
# -----

```

```

def get_hungarian_assignment(
    doctors,
    slots,
    cost_matrix
):

    slot_indices = hungarian_algorithm(
        cost_matrix
    )

    assignment = {}

    for doctor_index, slot_index in enumerate(
        slot_indices
    ):

        doctor = doctors[doctor_index]

```



```
    slot = slots[slot_index]

    assignment[doctor] = {
        "hospital": slot["hospital"],
        "slot": slot["slot"]
    }

    return assignment
```

```
# -----
# 9. GREEDY BASELINE
#
# Doctors are processed in the given order.
# Each doctor chooses the highest-ranked hospital
# that still has capacity.
# -----
```

```
def greedy_assignment(
    doctors,
    preferences,
    capacities
):

    remaining_capacity = capacities.copy()

    assignment = {}

    for doctor in doctors:

        for hospital in preferences[doctor]:

            if remaining_capacity[hospital] > 0:

                assignment[doctor] = {
                    "hospital": hospital
                }

                remaining_capacity[hospital] -= 1

                break

    return assignment
```

```
# -----
```

```
# 10. EVALUATE AN ASSIGNMENT
```

```
# -----
```

```
def evaluate_assignment(
    assignment,
    rank_lookup
):

    total_cost = 0

    first_choice_count = 0

    assigned_ranks = {}

    for doctor, result in assignment.items():

        hospital = result["hospital"]

        rank = rank_lookup[doctor][hospital]

        assigned_ranks[doctor] = rank

        total_cost += rank

        if rank == 1:
            first_choice_count += 1

    number_of_doctors = len(assignment)

    average_rank = (
        total_cost / number_of_doctors
    )

    first_choice_rate = (
        first_choice_count
        / number_of_doctors
    )

    return {
        "total_cost": total_cost,
        "average_rank": average_rank,
        "first_choice_count": first_choice_count,
        "first_choice_rate": first_choice_rate,
        "assigned_ranks": assigned_ranks
    }
```

```

# -----
# 11. GROUP RESULTS BY HOSPITAL
# -----

def group_by_hospital(
    assignment,
    capacities
):

    groups = {
        hospital: []
        for hospital in capacities
    }

    for doctor, result in assignment.items():

        hospital = result["hospital"]

        groups[hospital].append(doctor)

    return groups


# -----
# 12. PRINT ASSIGNMENT
# -----

def print_assignment(
    title,
    assignment,
    preferences,
    metrics,
    capacities
):

    print("\n")
    print("=" * 75)
    print(title)
    print("=" * 75)

    print(
        f"{'Doctor':<10}"
        f"{'Preference List':<30}"
        f"{'Assigned':<12}"
        f"{'Rank':<8}"
    )

```

```

)

print("-" * 75)

for doctor in doctors:

    hospital = assignment[doctor]["hospital"]

    rank = metrics[
        "assigned_ranks"
    ][doctor]

    preference_string = " > ".join(
        preferences[doctor]
    )

    print(
        f"{doctor:<10}"
        f"{preference_string:<30}"
        f"{hospital:<12}"
        f"{rank:<8}"
    )

print("\nMetrics")
print("-" * 40)

print(
    "Total rank cost:",
    metrics["total_cost"]
)

print(
    "Average rank:",
    round(
        metrics["average_rank"],
        3
    )
)

print(
    "First-choice doctors:",
    f"{metrics['first_choice_count']}"
    f"/{len(doctors)}"
)

print(

```

```

        "First-choice rate:",
        f"{metrics['first_choice_rate'] * 100:.1f}%"
    )

    print("\nAssignments by hospital")
    print("-" * 40)

    groups = group_by_hospital(
        assignment,
        capacities
    )

    for hospital, assigned_doctors in groups.items():

        print(
            f"Hospital {hospital} "
            f"(capacity {capacities[hospital]}): "
            f"{assigned_doctors}"
        )

```

```

# -----
# 13. COMPARE METHODS
# -----

```

```

def compare_methods(
    hungarian_metrics,
    greedy_metrics
):

    print("\n")
    print("=" * 75)
    print("HUNGARIAN VS GREEDY")
    print("=" * 75)

    print(
        f"{'Metric':<25}"
        f"{'Hungarian':<20}"
        f"{'Greedy':<20}"
    )

    print("-" * 65)

    print(
        f"{'Total rank cost':<25}"
        f"{hungarian_metrics['total_cost']:<20}"
    )

```

```

        f"{greedy_metrics['total_cost']:<20}"
    )

    print(
        f"{'Average rank':<25}"
        f"{hungarian_metrics['average_rank']:<20.3f}"
        f"{greedy_metrics['average_rank']:<20.3f}"
    )

    hungarian_first = (
        hungarian_metrics[
            "first_choice_rate"
        ] * 100
    )

    greedy_first = (
        greedy_metrics[
            "first_choice_rate"
        ] * 100
    )

    print(
        f"{'First-choice rate':<25}"
        f"{hungarian_first:<19.1f}%"
        f"{greedy_first:<19.1f}%"
    )

    improvement = (
        greedy_metrics["total_cost"]
        - hungarian_metrics["total_cost"]
    )

    print("\nImprovement in total cost:")

    print(
        f"{greedy_metrics['total_cost']} "
        f"- "
        f"{hungarian_metrics['total_cost']} "
        f"= {improvement}"
    )

```

```

# =====
# 14. MAIN PROGRAM
# =====

```

```

def main():

    print(
        "DOCTOR-HOSPITAL "
        "RANK-ORDER ASSIGNMENT"
    )

    print("=" * 75)

    # Validate input
    validate_input(
        doctors,
        preferences,
        capacities
    )

    # Build hospital slots
    slots = create_slots(
        capacities
    )

    print("\nHospital slots:")

    print(
        [
            slot["slot"]
            for slot in slots
        ]
    )

    # Build rank lookup
    rank_lookup = create_rank_lookup(
        preferences
    )

    # Build cost matrix
    cost_matrix = build_cost_matrix(
        doctors,
        slots,
        rank_lookup
    )

    # Print cost matrix
    print_cost_matrix(
        doctors,
        slots,

```

```

        cost_matrix
    )

# -----
# Hungarian Assignment
# -----

hungarian_result = (
    get_hungarian_assignment(
        doctors,
        slots,
        cost_matrix
    )
)

hungarian_metrics = (
    evaluate_assignment(
        hungarian_result,
        rank_lookup
    )
)

# -----
# Greedy Assignment
# -----

greedy_result = (
    greedy_assignment(
        doctors,
        preferences,
        capacities
    )
)

greedy_metrics = (
    evaluate_assignment(
        greedy_result,
        rank_lookup
    )
)

# -----
# Print Results
# -----

print_assignment(

```



```

        "HUNGARIAN ALGORITHM RESULT",
        hungarian_result,
        preferences,
        hungarian_metrics,
        capacities
    )

    print_assignment(
        "GREEDY BASELINE RESULT",
        greedy_result,
        preferences,
        greedy_metrics,
        capacities
    )

# -----
# Compare
# -----

    compare_methods(
        hungarian_metrics,
        greedy_metrics
    )

# -----
# Run program
# -----

if __name__ == "__main__":
    main()

```

DOCTOR-HOSPITAL RANK-ORDER ASSIGNMENT

Input validation passed.

Doctors: 18, Hospitals: 6, Total capacity: 18

Hospital slots:

['A1', 'A2', 'A3', 'B1', 'B2', 'B3', 'C1', 'C2', 'C3', 'D1', 'D2', 'D3', 'E1', 'E2', 'E3', 'F1', 'F2', 'F3']

COST MATRIX

Doctor	A1	A2	A3	B1	B2	B3	C1	C2	C3	D1	D2	D3	E1	E2	E3	F1	F2	F3
D1	1	1	1	6	6	6	4	4	4	3	3	3	5	5	5	2	2	2
D2	1	1	1	3	3	3	2	2	2	4	4	4	6	6	6	5	5	5
D3	3	3	3	5	5	5	4	4	4	6	6	6	1	1	1	2	2	2
D4	1	1	1	6	6	6	4	4	4	5	5	5	3	3	3	2	2	2
D5	2	2	2	4	4	4	5	5	5	6	6	6	1	1	1	3	3	3
D6	2	2	2	1	1	1	5	5	5	6	6	6	4	4	4	3	3	3
D7	6	6	6	3	3	3	4	4	4	2	2	2	1	1	1	5	5	5
D8	1	1	1	5	5	5	3	3	3	6	6	6	2	2	2	4	4	4
D9	1	1	1	3	3	3	5	5	5	4	4	4	2	2	2	6	6	6
D10	2	2	2	1	1	1	5	5	5	3	3	3	4	4	4	6	6	6
D11	6	6	6	2	2	2	3	3	3	4	4	4	1	1	1	5	5	5
D12	2	2	2	4	4	4	3	3	3	1	1	1	6	6	6	5	5	5
D13	5	5	5	2	2	2	3	3	3	1	1	1	6	6	6	4	4	4
D14	2	2	2	5	5	5	1	1	1	4	4	4	6	6	6	3	3	3
D15	3	3	3	5	5	5	1	1	1	4	4	4	2	2	2	6	6	6
D16	5	5	5	3	3	3	4	4	4	1	1	1	2	2	2	6	6	6
D17	4	4	4	5	5	5	1	1	1	2	2	2	3	3	3	6	6	6
D18	3	3	3	1	1	1	4	4	4	5	5	5	2	2	2	6	6	6

HUNGARIAN ALGORITHM RESULT

Doctor	Preference List	Assigned	Rank
D1	A > F > D > C > E > B	F	2
D2	A > C > B > D > F > E	A	1
D3	E > F > A > C > B > D	F	2
D4	A > F > E > C > D > B	F	2
D5	E > A > F > B > C > D	E	1
D6	B > A > F > E > C > D	B	1
D7	E > D > B > C > F > A	E	1
D8	A > E > C > F > B > D	A	1
D9	A > E > B > D > C > F	A	1

D10	B > A > D > E > C > F	B	1
D11	E > B > C > D > F > A	E	1
D12	D > A > C > B > F > E	D	1
D13	D > B > C > F > A > E	D	1
D14	C > A > F > D > B > E	C	1
D15	C > E > A > D > B > F	C	1
D16	D > E > B > C > A > F	D	1
D17	C > D > E > A > B > F	C	1
D18	B > E > A > C > D > F	B	1

Metrics

Total rank cost: 21
Average rank: 1.167
First-choice doctors: 15/18
First-choice rate: 83.3%

Assignments by hospital

Hospital A (capacity 3): ['D2', 'D8', 'D9']
Hospital B (capacity 3): ['D6', 'D10', 'D18']
Hospital C (capacity 3): ['D14', 'D15', 'D17']
Hospital D (capacity 3): ['D12', 'D13', 'D16']
Hospital E (capacity 3): ['D5', 'D7', 'D11']
Hospital F (capacity 3): ['D1', 'D3', 'D4']

=====

GREEDY BASELINE RESULT

=====

Doctor	Preference List	Assigned	Rank
D1	A > F > D > C > E > B	A	1
D2	A > C > B > D > F > E	A	1
D3	E > F > A > C > B > D	E	1
D4	A > F > E > C > D > B	A	1
D5	E > A > F > B > C > D	E	1
D6	B > A > F > E > C > D	B	1
D7	E > D > B > C > F > A	E	1
D8	A > E > C > F > B > D	C	3
D9	A > E > B > D > C > F	B	3
D10	B > A > D > E > C > F	B	1
D11	E > B > C > D > F > A	C	3
D12	D > A > C > B > F > E	D	1
D13	D > B > C > F > A > E	D	1
D14	C > A > F > D > B > E	C	1

D15	C > E > A > D > B > F	D	4
D16	D > E > B > C > A > F	F	6
D17	C > D > E > A > B > F	F	6
D18	B > E > A > C > D > F	F	6

Metrics

Total rank cost: 42
Average rank: 2.333
First-choice doctors: 11/18
First-choice rate: 61.1%

Assignments by hospital

Hospital A (capacity 3): ['D1', 'D2', 'D4']
Hospital B (capacity 3): ['D6', 'D9', 'D10']
Hospital C (capacity 3): ['D8', 'D11', 'D14']
Hospital D (capacity 3): ['D12', 'D13', 'D15']
Hospital E (capacity 3): ['D3', 'D5', 'D7']
Hospital F (capacity 3): ['D16', 'D17', 'D18']

=====			
HUNGARIAN VS GREEDY			
=====			
Metric	Hungarian	Greedy	

Total rank cost	21	42	
Average rank	1.167	2.333	
First-choice rate	83.3	%61.1	%

Improvement in total cost:
42 - 21 = 21

In []: